Debreceni SZC Mechwart András Gépipari és Informatikai Technikum

Szoftverfejlesztő és -tesztelő

Vizsgaremek

Mozijegy foglalási rendszer

Készítette: Árva Martina
Szabó Ádám Kornél

2026.

Tartalom

1. A szoftver célja és leírása 4
Fő funkciók 4
Technológiai stack 4
2. Rendszer követelmények és telepítés 4
Szükséges szoftverek 4
Backend indítása 4
Frontend indítása 5
Konfiguráció (appsettings.json) 5
3. Rendszerarchitektúra 5
Backend rétegek 5
Frontend struktúra 6
Kommunikációs folyamat 6
4. Adatbázis-modell 6
Táblák és mezök 6
Film 6
Terem 6
Szék 7
Vetítés 7
Felhasználó 7
Foglalás 7
Foglalthely 7
Kapcsolatok és indexek 8
5. Backend - REST API referencia 8
/api/Auth 8
/api/Film 8
/api/Vetites 8
/api/Foglalas 9
/api/FoglaltHely 9
/api/Szek 9
/api/Terem 9
/api/Felhasznalo 9
6. Frontend - kliens oldali komponensek 9
Oldalak (pages/) 10
Fontosabb komponensek (comps/) 10
Komponensek részletes leírása 10
NavBar.jsx 10
FilmGrid.jsx 10
SzekTerkep.jsx 10

Table.jsx és TableOne.jsx ... 11
AdminFilm.jsx ... 11
AdminVetites.jsx ... 11
Authcontext.jsx ... 11
API kommunikáció (service/api.js) ... 11
7. Hitelesítés és jogosultság kezelés ... 11
Regisztráció ... 11
Bejelentkezés ... 11
JWT tartalom ... 11
Admin jogosultság ... 12
8. Használati útmutató ... 12
Látogató (nem bejelentkezett felhasználó) ... 12
Regisztrált felhasználó ... 12
Adminisztrátor ... 12
9. Tesztelési dokumentáció ... 13
API végpont tesztek - fontosabb esetek ... 13
Felhasználói felület (manuális) tesztek ... 14

github link: https://github.com/Altaisss/moziprojekt.git

1. A szoftver célja és leírása

A Mozijegy Foglalási Rendszer egy webalapú alkalmazás, amely lehetővé teszi egy artmozi műsorának böngészését, filmek részleteinek megtekintését, vetítések listázását, valamint a helyfoglalást. A rendszer célja, hogy a látogatok egyszerűen és kényelmesen, mobilon is foglalhassanak jegyet a kiválasztott vetítésre.

A mozi küldetése, hogy bemutassa a különböző országok és régiók gazdag filmes kultúráját. A vetítéseken a filmek kizárólag eredeti nyelven, magyar felirattal láthatóak.

Fő funkciók

- Filmek böngészése, plakát és részletek megtekintése
- Vetítések listázása dátumok szerint szűrt táblázatban
- Interaktív szék-térkép szabad/foglalt/kiválasztott állapotokkal
- Felhasználói regisztráció és be jelentkezés (JWT token)
- Saját foglalások megtekintése és törlése a profil oldalon
- Admin felület filmek és vetítések kezeléshez (CRUD)
- Vetítések időpontjainak ellenőrzése admin oldalon
- Reszponzív, mobil- és asztalbarát felület

Technológiai stack

Réteg	Technológia	Verzió / megjegyzés
Backend API	ASP.NET Core	8.0, C#
ORM	Entity Framework Core	SQLite adatbázissal
Hitelesítés	JWT Bearer token	BCrypt jelszó hashelés
Frontend	React	18, Vite build tool
Routing (kliens)	React Router	v6
HTTP kliens	Fetch API	service/api.js rétegen át
Adatbázis	SQLite	mozi.db fájl
Kép feltöltés	ASP.NET multipart/form-data	wwwroot/Images

2. Rendszer követelmények és telepítés

Szükséges szoftverek

- .NET 8 SDK (https://dotnet.microsoft.com)
- Node.js 18+ és npm
- Git

Backend indítása

A backend egy ASP.NET Core 8 Web API projekt. Indításához:

```
cd backend
dotnet restore
dotnet ef database update   # adatbázis-migrációk lefuttatása
```

```
dotnet run
```

A szerver alapesetben a https://localhost:7220 címen éri el az API-t. Az appsettings.json fájlban konfigurálható a kapcsolati string és a JWT Secret.

Frontend indítása

A frontend Vite + React projektként van felszerelve:

```
cd mozifront
npm install
npm run dev
```

A fejlesztői szerver általában a http://localhost:5173 címen indul el. A BASE_URL konstans az api.js fájlban mutat a backend URL-re (https://localhost:7220/api).

Konfiguráció (appsettings.json)

```
{
  "ConnectionStrings": {
    "DefaultConnection": "Data Source=mozi.db"
  },
  "Jwt": {
    "Key": "kulcs",
    "Issuer": "MoziApi",
    "Audience": "MoziClient"
  }
}
```

3. Rendszerarchitektúra

Az alkalmazás klasszikus hámrétegű (three-tier) architektúrán alapszik: kliens oldali React SPA, szerver oldali ASP.NET Core REST API, és egy SQLite adatbázis. A backend belső felépítése tovább tagolódik rétegekre a tiszta kód elveinek megfelelően.

Backend rétegek

Réteg	Felelősség	Fontosabb fájlok
Controller	HTTP kérések fogadása, válaszok visszaküldése	FilmController.cs, VetitesController.cs, FoglalasController.cs, AuthController.cs, stb.
Service	Üzleti logika, validáció, adatalakítás	FilmService.cs, VetitesService.cs, AuthService.cs, stb.
Repository	Adatbázis műveletek absztrakciója	Repository.cs, FilmRepository.cs, VetitesRepository.cs, stb.
DbContext	EF Core adatbázis-kapcsolat, modell-konfiguráció	MoziDbContext.cs
DTO	Kliens felé menő / érkező adat struktúrák	FilmDto.cs, VetitesDto.cs, FoglalasDto.cs, stb.
Service	JWT generálás, kép fájl mentese	JwtService.cs, FileService.cs

Frontend struktúra

Könyvtár / fájl	Szerepe
pages/	Teljes oldalak (Home, Vetitesek, Foglalas, Profile, AdminPage, Filmdetails)
comps/	Újrahasznosítható komponensek (NavBar, FilmGrid, SzekTerkep, Table, AdminFilm, stb.)
service/api.js	Összes backend API-hívás centralizálva (fetch wrapperek)
comps/Authcontext.jsx	React Context - globális bejelentkezési állapot, JWT kezelése
comps/Adminroute.jsx	Védett útvonal - csak adminnak engedett oldalak
css/	Komponensenként CSS stílus fájlok

Kommunikációs folyamat

A React SPA a Fetch API-n keresztül kommunikál a backenddel. A JWT token a localStorage-ban tárolódik. Minden hitelesítést igénylő kéréshez az Authorization: Bearer <token> fejléc kerül a kérésbe. Az api.js-ben lévő getToken() segéd függvény ezt automatikusan kezeli.

4. Adatbázis-modell

Az Adatbázis SQLite alapú, Entity Framework Core Code-First migrációkkal kezelt. Az Adatbázis fájl neve mozi.db, a projekt gyökér mappájában található.

Táblák és mezők

Film

Mező	Típus	Leírás
Id	INT PK	Egyedi azonosító
Cim	VARCHAR(200)	A film címe (kötelező)
Rendezo	VARCHAR(200)	Rendező neve (kötelező)
Hossz	INT	Film hossza percben (1-500)
Leírás	VARCHAR(2000)	Opcionális szinopszis
KepUrl	VARCHAR	Relatív URL a plakát képhez (/Images/...)

Terem

Mező	Típus	Leírás
Id	INT PK	Egyedi azonosító
TeremNev	VARCHAR	Terem neve (pl. Terem 1)

Szék

Mező	Típus	Leírás
Id	INT PK	Egyedi azonosító
Sor	INT	Sor száma
Szam	INT	Ülőhely száma a soron belül
Oldal	CHAR(1)	B = bal, J = jobb (folyosó két oldala)
TeremId	INT FK	Kapcsolat a Terem táblával

Vetítés

Mező	Típus	Leírás
Id	INT PK	Egyedi azonosító
Idopont	DATETIME	Vetítés kezdete
FilmId	INT FK	A vetített film
TeremId	INT FK	A vetítésnek helyet adó terem
JegyAr	INT	Jegyár forintban
Nyelv	VARCHAR	Vetítés nyelve (pl. japán, francia)

Felhasználó

Mező	Típus	Leírás
Id	INT PK	Egyedi azonosító
Nev	VARCHAR	Teljes név
Email	VARCHAR(200) UNIQUE	E-mail cím (bejelentkezéshez)
Jelszo	VARCHAR(256)	BCrypt hashelt jelszó
IsAdmin	BOOL	Admin jogosultság jelzője

Foglalás

Mező	Típus	Leírás
Id	INT PK	Egyedi azonosító
FelhasznaloId	INT FK	A foglaló felhasználó

Foglalthely

Mező	Típus	Leírás
Id	INT PK	Egyedi azonosító
SzekId	INT FK	A lefoglalt szék
FoglalasId	INT FK	A tartozó foglalás
VetitesId	INT FK	A vetítés, amelyre foglaltak

Kapcsolatok és indexek

- Foglalthely(VetitesId, SzekId) - UNIQUE index: egy szék egy vetítésen csak egyszer foglalható
- Felhasznalo(Email) - UNIQUE index: e-mail cím nem ismétlődhet
- Vetites(Idopont) - gyors lekérdezésekhez (mai műsor, közelgő vetítések)
- Vetites(FilmId, Idopont) és Vetites(TeremId, Idopont) - összetett indexek
- Foglalas(FelhasznaloId) - saját foglalások gyors listázásához
- Foglalthely törlése foglalás törlésekor: CASCADE delete beállítva

5. Backend - REST API referencia

Az API alap URL-je: https://localhost:7220/api. A végpontok RESTful elvek szerint épülnek fel. A [Authorize] attribútummal jelölt végpontok érvényes JWT tokent igényelnek az Authorization: Bearer <token> fejlécben.

/api/Auth

Metódus	Útvonal	Leírás	Hitelesítés
POST	/api/Auth/register	Új felhasználó regisztrálása	Nem szükséges
POST	/api/Auth/login	Bejelentkezés, JWT token visszaadása	Nem szükséges

/api/Film

Metódus	Útvonal	Leírás	Hitelesítés
GET	/api/Film	Összes film listázása	Nem szükséges
GET	/api/Film/{id}	Egy film adatai	Nem szükséges
POST	/api/Film	Új film létrehozása (kép feltöltéssel)	Admin token
PUT	/api/Film/{id}	Film adatainak frissítése	Admin token
DELETE	/api/Film/{id}	Film törlése	Admin token

/api/Vetites

Metódus	Útvonal	Leírás	Hitelesítés
GET	/api/Vetites	Összes vetítés	Nem szükséges
GET	/api/Vetites/{id}	Egy vetítés adatai	Nem szükséges
POST	/api/Vetites	Új vetítés létrehozása	Admin token
PUT	/api/Vetites/{id}	Vetítés módosítása	Admin token
DELETE	/api/Vetites/{id}	Vetítés törlése	Admin token

/api/Foglalas

Metódus	Útvonal	Leírás	Hitelesítés
GET	/api/Foglalas	Felhasználó foglalásainak listája	Bearer token
POST	/api/Foglalas	Új foglalás létrehozása	Bearer token
DELETE	/api/Foglalas/{id}	Foglalás törlése	Bearer token (saját)

/api/FoglaltHely

Metódus	Útvonal	Leírás	Hitelesítés
GET	/api/FoglaltHely	Összes foglalt hely (vetítés szűréshez)	Nem szükséges
POST	/api/FoglaltHely	Szék lefoglalása egy vetítésre	Bearer token
DELETE	/api/FoglaltHely/{id}	Foglalt hely törlése	Bearer token

/api/Szek

Metódus	Útvonal	Leírás	Hitelesítés
GET	/api/Szek	Összes szék	Nem szükséges
GET	/api/Szek/{id}	Egy szék adatai	Nem szükséges

/api/Terem

Metódus	Útvonal	Leírás	Hitelesítés
GET	/api/Terem	Összes terem	Nem szükséges
GET	/api/Terem/{id}	Egy terem adatai	Nem szükséges
POST	/api/Terem	Új terem létrehozása	Admin token

/api/Felhasznalo

Metódus	Útvonal	Leírás	Hitelesítés
GET	/api/Felhasznalo	Bejelentkezett felhasználó adatai	Bearer token
PUT	/api/Felhasznalo/{id}	Adatok frissítése	Bearer token
DELETE	/api/Felhasznalo/{id}	Fiók törlése	Bearer token

6. Frontend - kliens oldali komponensek

A frontend React 18 alapú Single Page Application (SPA), amelyet a Vite build eszközzel futtatunk. A React Router v6 kezeli az oldalak közti navigációt.

Oldalak (pages/)

Útvonal	Komponens	Leírás
/	Home.jsx	Főoldal: filmek rácsban, üdvözlő szöveg
/vetitesek	Vetitesek.jsx	Összes vetítés táblázatosan, dátum szerint szűrve
/vetitesek/:id	Foglalas.jsx	Szék térkép adott vetítéshez, foglalási logika
/filmek/:id	Filmdetails.jsx	Film részletei + az adott film vetítései
/profile	Profile.jsx	Bejelentkezés / regisztráció / saját foglalások
/admin	AdminPage.jsx	Admin CRUD felület (védett, csak adminnak)

Fontosabb komponensek (comps/)

Komponens	Leírás
NavBar.jsx	Navigációs sáv: Home, Vetítések, (Admin), Profil linkek. Admin link csak admin bejelentkezésnél látszik.
FilmGrid.jsx	Filmek rácsban, poszterekkel és címekkel. Kattintásra a film részletes lapjára navigál.
SzekTerkep.jsx	Interaktív szék térkép: sorok / bal-jobb oldal / folyosó. Szabad, foglalt, kiválasztott állapotok.
Table.jsx	Vetítési műsor táblázat: filmek soraiban, dátumok oszlopaiban időpontok linkekkel.
TableOne.jsx	Ugyanolyan táblázat, de egyetlen filmhez (Filmdetails oldalon használt).
AdminFilm.jsx	Film kezelő admin panel: lista + form (hozzáadás/szerkesztés/törlés + kép feltöltés).
AdminVetites.jsx	Vetítés kezelő admin panel: lista + form + ütközés-detektor logika.
Authcontext.jsx	React Context: bejelentkezési állapot, JWT dekódolás, be/ki jelentkezés/regisztráció metódusok.
Adminroute.jsx	Route guard: ha nem admin, visszairányít a főoldalra.

Komponensek részletes leírása

NavBar.jsx

A navigációs sáv az alkalmazás tetején jelenik meg minden oldalon. A React Router Link komponenseit használja az oldalak közti navigációhoz az oldal újra betöltése nélkül. Az useAuth() hook segítségével olvassa a bejelentkezési állapotot: ha a felhasználó be van jelentkezve ÉS adminisztrátor, megjelenik az Admin menüpont is. A navigációs elemek CSS-sel reszponzívan igazodnak mobil nézeten is.

FilmGrid.jsx

A főoldalon megjelenített filmrácsot valósítja meg. A filmek több oszlopos rácsban jelennek meg, mindegyik egy kattintható kártyaként (poster-card). Minden kártyán látszik a film posztere és a film címe. Ha a kép nem tölthető be, az onError esemény kezelő egy placeholder képre vált. A Link komponens a /filmek/:id útvonalra navigál kattintásra.

SzekTerkep.jsx

Az interaktív szék térkép a foglalási folyamat központi eleme. A székek sor és oldal információt hordoznak (B = bal, J = jobb oldal, folyosóval elválasztva). Három vizuális állapotot különböztet

meg: szabad (fehér), foglalt (piros, letiltott), és kiválasztott (kék). A felhasználó több széket is kiválaszthat egyszerre - a kiválasztott székek egy local state tömbben tárolódnak. A Foglalás gomb csak akkor aktív, ha legalább egy szék ki van jelölve. Sikeres foglalás után a kiválasztás visszaáll, az új foglalt székek azonnal látszanak.

Table.jsx és TableOne.jsx

A Table komponens az összes film vetítéseit jeleníti meg egy dinamikus rácsban: a sorok a filmeket, az oszlopok a dátumokat képviselik. Az oszlop fejlécek automatikusan a vetítésekből származtatott egyedi dátumok. Minden cellában az adott filmhez és dátumhoz tartozó vetítési időpontok linkként jelennek meg. A TableOne ugyanezt a logikát valósítja meg egyetlen film számára a Filmdetails oldalon.

AdminFilm.jsx

A filmek adminisztrációját megvalósító panel két részből áll: egy űrlap új film hozzáadásához vagy meglevő szerkesztéséhez, és egy táblázat az összes filmről. Az űrlap kép feltöltést is támogat (multipart/form-data). Szerkesztési módban a meglévő poszter előnézete megjelenik. Mentés/Hozzáadás gomb megnyomása után a lista automatikusan frissül. Törlésnél megerősítést kér a rendszer és a kép is törlődik a szerverről.

AdminVetites.jsx

A vetítés adminisztrációt megvalósító panel kiegészítve egy ütközést figyelő algoritmussal. Mielőtt mentés történne, a checkConflict() függvény megvizsgálja, hogy a kiválasztott teremben az adott időpontban (a film hosszával számolt idősávon belül) már van-e másik vetítés. Ütközés esetén figyelmeztetés jelenik meg a terem nevével, az ütköző film címével és időpontjával.

Authcontext.jsx

A React Context API alapú globális hitelesítési állapot kezelő. Inicializáláskor beolvassa a localStorage-ból a JWT tokent (ha van), és jwt-decode könyvtárral kibontja belőle a felhasználó adatait (id, e-mail, név). Exportál három művelet metódust: Bejelentkezés(), regisztracio() és kijelentkezes(). Az isAdmin() segéd függvény a tokenből olvassa ki az admin jogot, és bárhol importálható anélkül, hogy az egész Contextet kellene használni.

API kommunikáció (service/api.js)

Az api.js fájl tartalmaz minden fetch-hívást. A getToken() segéd függvény automatikusan illeszti a JWT tokent az Authorization fejlécbe ahol szükséges. A handleResponse() egységesen kezeli a HTTP hiba válaszokat.

7. Hitelesítés és jogosultság kezelés

Regisztráció

POST /api/Auth/register végpontra küldött kéréssel. A jelszó BCrypt.Net könyvtárral hashelve tárolódik. Az e-mail egyediségének ellenőrzése adatbázis szinten is biztosított (UNIQUE index).

Bejelentkezés

POST /api/Auth/login végpont. Siker esetén egy JWT token kerül visszaküldésre. A token 8 óráig érvényes (a lejárat ideje az exp claimben tárolva). A kliensen localStorage-ban tárolódik a token.

JWT tartalom

Claim	Érték	Használat
sub	Felhasznalo ID (int)	Azonosítás kliensen és szerveren
email	E-mail cím	Megjelenítés

Claim	Érték	Használat
nev	Teljes név	Megjelenítés
isAdmin	True / False (string)	Admin útvonal engedélyezése
exp	Unix timestamp	Token lejárata (8 óra)

Admin jogosultság

Az isAdmin claim értéke alapján az Adminroute.jsx komponens engedi vagy megtagadja a /admin útvonal elérést. Szerver oldalon a [Authorize] attribútum védi a végpontokat. Az admin flag az adatbázisban tárolódik (Felhasznalo.IsAdmin mező).

8. Használati útmutató

Látogató (nem bejelentkezett felhasználó)

- A főoldalon megtalálható az összes aktuális film poszterekkel.
- A Vetítések oldalon dátum és film szerint szűrt táblázatban böngészhetők a vetítések.
- Bármely filmre kattintva elérhető a film részletes oldala (leírás, rendező, hossz, vetítések).
- Foglaláshoz bejelentkezés szükséges - a rendszer a Profil oldalra irányítja a felhasználót.

Regisztrált felhasználó

- Regisztráció: név, e-mail, jelszó megadása (Profil oldal -> Regisztráció gomb).
- Bejelentkezés után a JWT token tárolásra kerül, a navigáció frissül.
- Vetítés kiválasztása után a szék térképen láthatóak a szabad (fehér) és foglalt (piros) helyek.
- Egy vagy több szék kiválasztása (kék), majd a Foglalás gomb megnyomása.
- Siker eseten visszajelzés jelenik meg, a székek azonnal foglalttá válnak.
- A Profile oldalon megtekinthető az összes saját foglalás (film, terem, időpont, székek).
- Foglalás törölhető a 'Foglalás törlése' gombbal (megerősítést kér).

Adminisztrátor

- Bejelentkezés után az Admin menüpont megjelenik a navigációban.
- Film hozzáadása: cím, rendező, hossz, leírás, poszter feltöltése.
- Film szerkesztése: meglevő kép megtartása vagy csere.
- Film törlése: megerősítés után a kép is törlődik a szerverről.
- Vetítés hozzáadása: film, terem és időpont kiválasztása, jegyár, nyelv megadása.
- Ütközés-detektor: ha a kijelölt teremben az adott időpontban mar van vetítés, figyelmeztetés.
- Vetítés szerkesztése és törlése is elérhető.

9. Tesztelési dokumentáció

A tesztelést manuális végpont tesztelés (Swagger / HTTP klienssel) és böngészős funkcionalitás tesztelés formájában végeztük.

API végpont tesztek - fontosabb esetek

Teszteset	Végpont	Bemenet	Várt eredmény	Állapot
Sikeres regisztráció	POST /api/Auth/register	Érvényes név, email, jelszó	200 OK, 'Sikeres regisztráció'	OK
Ismételt e-mail regisztráció	POST /api/Auth/register	Már létező e-mail cím	409 Conflict	OK
Sikeres Bejelentkezés	POST /api/Auth/login	Helyes e-mail és jelszó	200 OK, JWT token	OK
Hibás Bejelentkezés	POST /api/Auth/login	Helytelen jelszó	401 Unauthorized	OK
Filmek listázása	GET /api/Film	-	200 OK, JSON tömb	OK
Nem létező film	GET /api/Film/9999	-	404 Not Found	OK
Film létrehozása (admin)	POST /api/Film	FormData: cím, rendező, hossz, kép	200 OK, FilmResponse	OK
Film létrehozása (nem admin)	POST /api/Film	FormData + felhasználói token	403 Forbidden	OK
Vetítések listázása	GET /api/Vetites	-	200 OK, JSON tömb	OK
Foglalás létrehozása (token)	POST /api/Foglalas	felhasznaloId	201 Created, FoglalasResponse	OK
Foglalás token nélkül	POST /api/Foglalas	-	401 Unauthorized	OK
Székfoglalás	POST /api/FoglaltHely	szekId, foglalasId, vetitesId	201 Created	OK
Már foglalt szék újra foglalása	POST /api/FoglaltHely	Ismételt szék és vetítés	409 Conflict	OK
Foglalás törlése (saját)	DELETE /api/Foglalas/{id}	Saját foglalás ID-ja	204 No Content	OK
Foglalás törlése (másik felhasználó)	DELETE /api/Foglalas/{id}	Más felhasználó ID-ja	403 Forbidden	OK

Felhasználói felület (manuális) tesztek

Teszteset	Lépés	Várt eredmény	Állapot
Főoldal betöltése	Alkalmazás megnyitása	Filmek rács megjelenik	OK
Film részlet oldal	Film poszterre kattintás	Leírás, rendező, vetítések táblázata	OK
Vetítések oldal	Vetítések menü	Dátum oszlopos műsor táblázat	OK
Regisztráció	Profile -> Regisztráció	Sikeres üzenet, majd Bejelentkezési form	OK
Bejelentkezés	Profile -> Bejelentkezés	Navigáció frissül, Admin latszik, ha van jog	OK
Szék térkép megnyitása	Vetítés időpontjára kattintás	Interaktív szék térkép betolt	OK
Szék kiválasztása	Szabad székre kattintás	Kék kiemelés jelenik meg	OK
Foglalás elvégzése	Foglalás gomb megnyomása	Alert: 'Sikeres foglalás! N szék lefoglalva'	OK
Foglalt szék tiltása	Foglalt székre kattintás	Gomb disabled, kattintás nem lehetséges	OK
Saját foglalások	Profil oldal bejelentkezve	Foglalások listája: film, terem, időpont, helyek	OK
Foglalás törlése	Foglalás törlése gomb	Megerősítés, majd foglalás eltűnik a listából	OK
Admin - film hozzáadás	Admin -> Új film form	Film megjelenik a táblázatban és a főoldalon	OK
Admin - vetítés ütközés	Admin -> Ütköző időpont megadás	Alert: ütközésről szóló hiba üzenet	OK
Kijelentkezés	Kijelentkezés gomb	Token törlődik, navigáció visszaáll	OK
Reszponzív megjelenítés	Mobil nézet (375px)	Navigáció és szék térkép használható	OK